A PROJECT REPORT

On

Pneumonia Detection: Pushing the Boundaries of Human Ability with Deep Learning

By: Shahista Tamkeen

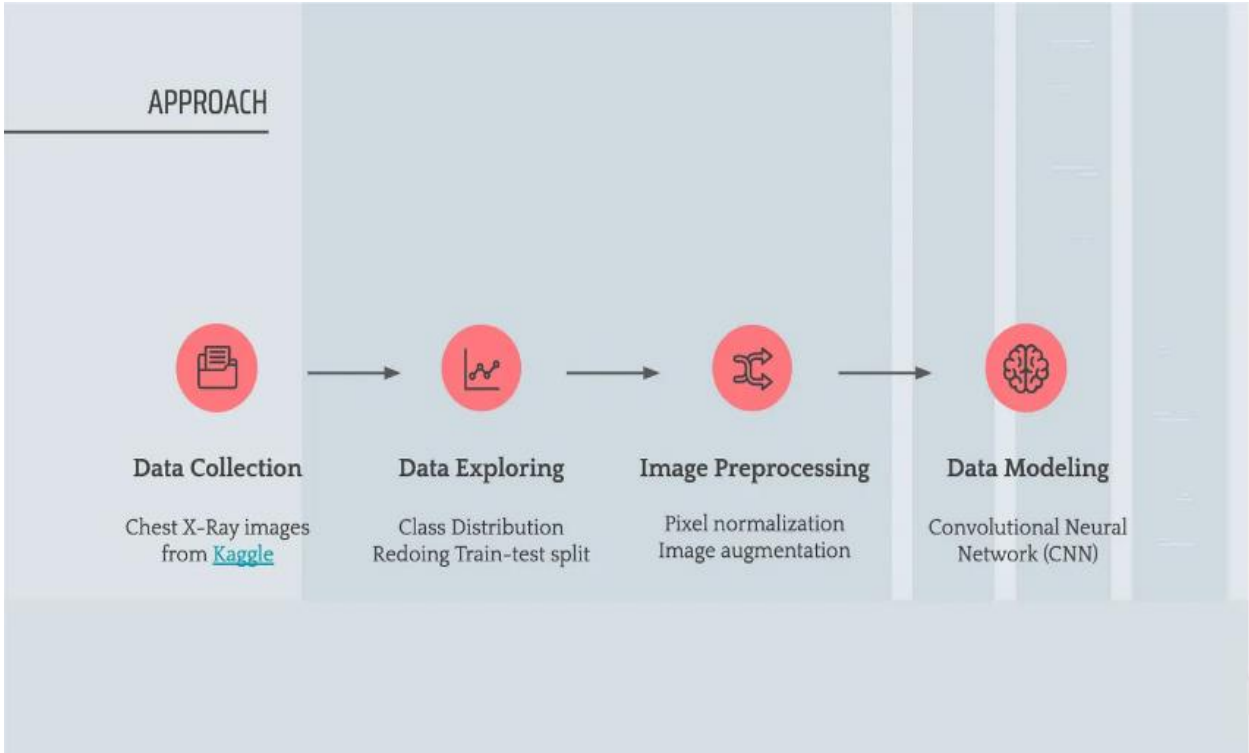
INTRODUCTION

Pneumonia is a severe lung infection that poses a significant health risk, especially to children and the elderly. Early and accurate diagnosis is crucial but often hampered by human error, subjective interpretations, and delays associated with traditional methods using chest X-rays. This project leverages the power of Convolutional Neural Networks (CNNs), a deep learning technique specifically designed for image analysis, to automate and enhance pneumonia detection. By analyzing chest X-ray images, the system delivers fast, precise, and reliable diagnoses, minimizing the risks of misdiagnosis and delays.

The integration of CNNs with healthcare introduces a transformative approach to addressing global medical challenges. The proposed solution combines data preprocessing, advanced image modeling, and scalable deployment techniques to create an efficient pipeline. It not only improves diagnostic accuracy but also reduces the workload of healthcare professionals. Furthermore, deploying this model using modern frameworks and cloud platforms makes it accessible for real-world applications, including mobile and embedded systems.

This project underscores the potential of artificial intelligence in revolutionizing healthcare, particularly in critical areas such as early disease detection. By automating pneumonia diagnosis, it paves the way for faster medical decisions, better patient outcomes, and a significant step toward AI-powered healthcare advancements.

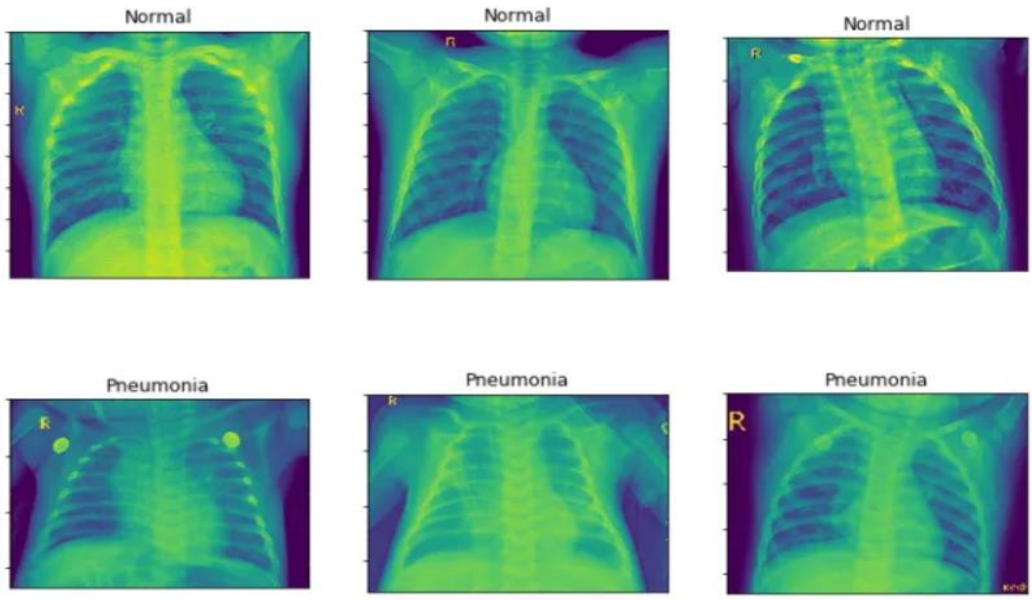
METHODOLOGY



Methodology of our project mainly involves 4 steps:

1. Data Collection:

The dataset for this project is sourced from Kaggle, comprising chest X-ray images labeled as normal or pneumonia cases. This dataset provides the foundation for training and validating the model. It contains a variety of images that represent different cases, ensuring diversity and robustness. The collection process involves downloading and organizing the dataset into appropriate directories for training, validation, and testing. Using Kaggle ensures access to a reliable, pre-labeled dataset widely used in medical imaging projects, allowing the project to focus on developing and refining the detection model rather than manually curating data.



Chest X-Ray images of patients from a medical center in Guangzhou, China (Dataset: Kaggle)

2. Data Exploration:

Data exploration involves analyzing the dataset to understand its structure and balance. This includes checking the class distributions to ensure an equal number of images for normal and pneumonia cases, as imbalances could skew model predictions. Additionally, train-test splits are reorganized to ensure fair representation across all data subsets. Visual inspections of the images help identify anomalies, such as corrupted files or inconsistencies, that need to be addressed. This step ensures the dataset is clean, balanced, and well-prepared for reliable model evaluation.

3. Image Preprocessing:

Image preprocessing prepares the data for efficient and accurate modeling. Pixel normalization standardizes pixel intensity values to a common range (e.g., 0 to 1), improving computational efficiency and stabilizing model training. Data augmentation artificially increases the dataset size by applying transformations such as flipping, rotating, and brightness adjustments. These techniques help create diverse training examples, making the model more robust to variations in input images. Preprocessing

ensures that the images are uniform, reducing noise and enhancing the model's ability to generalize to unseen data.

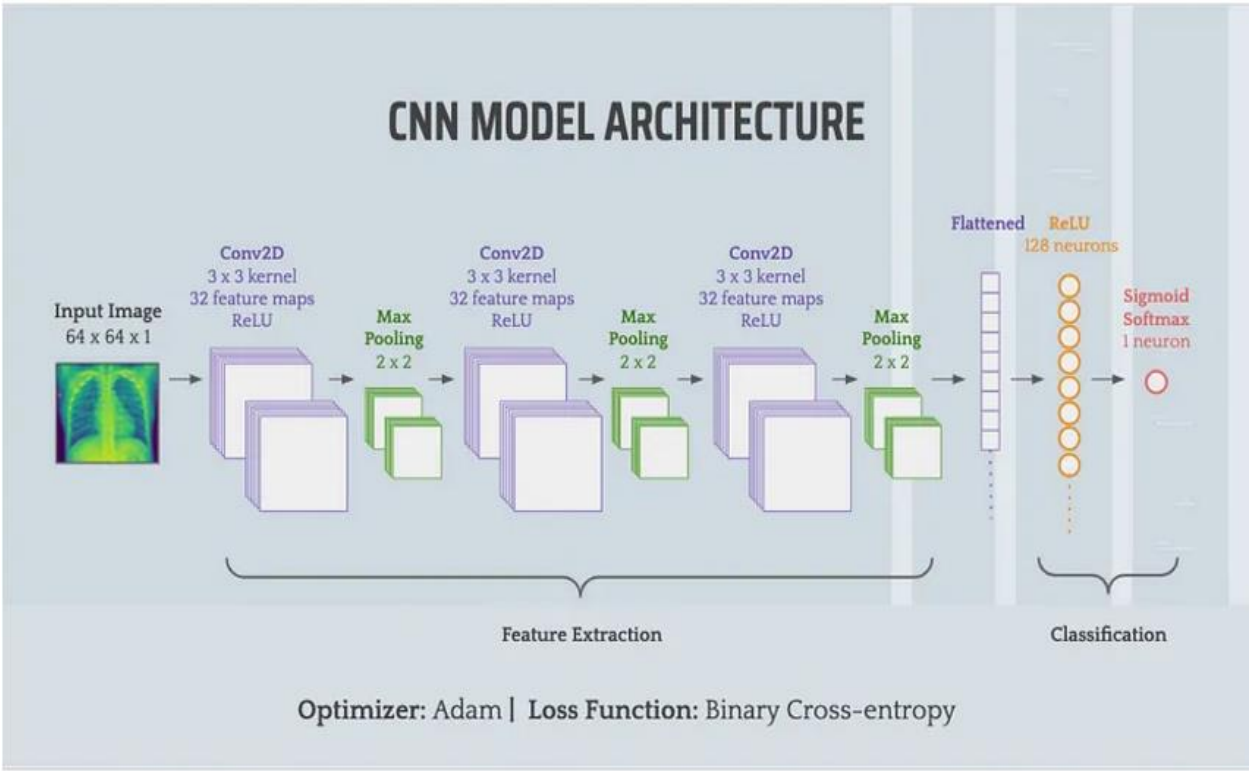
- **Pixel Normalization:** For most image data, pixel values are integers with values between 0 to 255. As neural networks process inputs using small weight values, inputs with larger values slow down the learning process. Normalizing pixels would shrink the integers to values between 0 to 1, hence it’s a good practice to normalize pixels to ensure efficient computation.

- **Data Augmentation:** we’ve also applied some augmentations to the images: horizontal flip, zoomed-in, and sheer. Data augmentation is a good practice to not only to add more data to the existing dataset but also add minor alterations, a.k.a. diversity, to avoid the model from overfitting to training data.

4. Data Modeling:

The project uses Convolutional Neural Networks (CNNs), a deep learning architecture tailored for image analysis. The model consists of convolutional layers for feature extraction, pooling layers to reduce dimensionality, a flattening layer to transition to dense layers, and fully connected layers for final predictions. Each convolutional layer applies filters to identify patterns such as edges or textures, while pooling layers reduce spatial complexity. The architecture is carefully designed to balance depth and computational efficiency, enabling the model to learn hierarchical features critical for distinguishing pneumonia from normal cases. This step transforms raw data into actionable diagnostic predictions.

MODEL ARCHITECTURE



On a high level, we have 3 convolutional 2D layers, with each followed by a max pooling layer. After being flattened, it is fed into a fully connected layer with ReLU activation. Then in the end, since this is a binary classification, we have the output layer using sigmoid softmax activation to predict an output probability, which can be converted to class values based on our threshold.

SYSTEM REQUIREMENTS

SOFTWARE REQUIREMENTS:

- **Operating System** **:** Windows 10
- **Programming Language** **:** Python
- **Tools** **:** Google Colab, Visual Studio Code

HARDWARE REQUIREMENTS:

- **Processor** **:** Intel Core i7
- **RAM** **:** Minimum 8 GB

SYSTEM ANALYSIS

ALGORITHM USED:

1. Convolutional Neural Network (CNN) Architecture:

- **Conv2D Layers**: Extract spatial features from images using convolution filters.
- **MaxPooling2D Layers**: Reduce spatial dimensions to avoid overfitting and reduce computation.
- **Flatten Layer**: Converts the 2D feature maps into a 1D vector for the dense layers.
- **Dense Layers**: Perform classification by connecting all neurons for final predictions.
- **Activation Functions**:
    - **ReLU**: Used in hidden layers to introduce non-linearity.
    - **Sigmoid**: Used in the output layer for binary classification.

2. Optimization Algorithm:

- **Adam Optimizer**: Used to optimize the model's weights by adjusting learning rates adaptively during training.

3. Regularization Techniques:

- **Dropout (Commented)**: Randomly drops neurons during training to prevent overfitting.
- **Batch Normalization**: (Not explicitly used here but is imported) Normalizes activations to stabilize learning.

METRICS FOR EVALUATION:

- **AUC (Area Under the Curve)**: Measures the performance of the model across thresholds.
- **Confusion Matrix Metrics**:
    - **Accuracy**: Overall correctness.
    - **Precision**: Proportion of true positives among predicted positives.
    - **Recall (Sensitivity)**: Proportion of actual positives identified.
    - **Specificity**: Proportion of actual negatives identified.

- **F1 Score**: Harmonic mean of precision and recall.

DATASET USED

The dataset used in the above code is the **Chest X-Ray dataset**, which contains chest X-ray images categorized into two classes: **Normal** and **Pneumonia**. This dataset is widely available on platforms like **Kaggle** and typically includes three subsets:

1. **Train Set**: Used for training the model.
2. **Validation Set**: Used to evaluate the model's performance during training.
3. **Test Set**: Used to assess the final model's performance.

The dataset structure referenced in the code suggests the following directory organization:

- train/: Contains subdirectories for "Normal" and "Pneumonia" cases.
- val/: Contains a smaller validation set with similar categories.
- test/: Contains subdirectories for "Normal" and "Pneumonia" to evaluate the model.

Each image is processed in grayscale mode and resized to a specified dimension (64x64 pixels). Data augmentation (e.g., rescaling, shear, zoom, and horizontal flipping) is applied to enhance the training set. This dataset is commonly used for building image classification models to detect pneumonia using deep learning techniques.

CODING

Import necessary libraries

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn.metrics import auc
import keras
import tensorflow as tf
from keras import backend as K
from keras import metrics
from keras.regularizers import l2
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization, InputLayer, Activation
from keras.preprocessing.image import ImageDataGenerator
from keras.metrics import AUC
from keras.optimizers import Adam
from keras.callbacks import EarlyStopping
from sklearn.metrics import classification_report, confusion_matrix
```

Set random seed

```
# Set a seed value

seed_value= 42
```

```
# 1. Set `PYTHONHASHSEED` environment variable at a fixed value

import os

os.environ['PYTHONHASHSEED']=str(seed_value)

# 2. Set `python` built-in pseudo-random generator at a fixed value

import random

random.seed(seed_value)

# 3. Set `numpy` pseudo-random generator at a fixed value

np.random.seed(seed_value)

# 4. Set `tensorflow` pseudo-random generator at a fixed value

tf.set_random_seed(seed_value)

# 5. For layers that introduce randomness like dropout, make sure to set seed values

#model.add(Dropout(0.25, seed=seed_value))

#6 Configure a new global `tensorflow` session

session_conf = tf.ConfigProto(intra_op_parallelism_threads=1, inter_op_parallelism_threads=1)

sess = tf.Session(graph=tf.get_default_graph(), config=session_conf)

K.set_session(sess)
```

Generate batches of image data (train, validation, and test) with data augmentation

```
## Set file paths to image files
```

```
project_path = "/Users/hellojenny/Metis/Metis-Bootcamp/Projects/Project 5 - Image Prediction using CNN"

train_path = project_path + "/chest_xray/train/"

val_path = project_path + "/chest_xray/val/"

test_path = project_path + "/chest_xray/test/"

## Set up hyperparameters that will be used later

hyper_dimension = 64

hyper_batch_size = 128

hyper_epochs = 100

hyper_channels = 1

hyper_mode = 'grayscale'

## Generate batches of image data (train, validation, and test) with data augmentation

train_datagen = ImageDataGenerator(rescale=1.0/255.0,

shear_range = 0.2,

zoom_range = 0.2,

horizontal_flip = True)

val_datagen = ImageDataGenerator(rescale=1.0/255.0)

test_datagen = ImageDataGenerator(rescale=1.0/255.0)

train_generator = train_datagen.flow_from_directory(directory = train_path,

target_size = (hyper_dimension, hyper_dimension),

batch_size = hyper_batch_size,

color_mode = hyper_mode,

class_mode = 'binary',

seed = 42)
```

```
val_generator = val_datagen.flow_from_directory(directory = val_path,
target_size = (hyper_dimension, hyper_dimension),
batch_size = hyper_batch_size,
class_mode = 'binary',
color_mode = hyper_mode,
shuffle=False,
seed = 42)
test_generator = test_datagen.flow_from_directory(directory = test_path,
target_size = (hyper_dimension, hyper_dimension),
batch_size = hyper_batch_size,
class_mode = 'binary',
color_mode = hyper_mode,
shuffle=False,
seed = 42)
test_generator.reset()
```

Train the model

```
cnn = Sequential()
cnn.add(InputLayer(input_shape=(hyper_dimension, hyper_dimension, hyper_channels)))
cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu'))
cnn.add(MaxPooling2D(pool_size=(2,2)))
cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu'))
cnn.add(MaxPooling2D(pool_size=(2,2)))
```

```
cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu'))

cnn.add(MaxPooling2D(pool_size=(2,2)))

cnn.add(Flatten())

cnn.add(Dense(activation='relu', units=128))

cnn.add(Dense(activation='sigmoid', units=1))

cnn.compile(optimizer= 'adam', loss='binary_crossentropy', metrics=[AUC()])

cnn_model = cnn.fit_generator(train_generator,

steps_per_epoch = len(train_generator),

epochs = 200,

validation_data = val_generator,

validation_steps = len(val_generator),

verbose=2)
```

Create a function that outputs metrics and charts

- **Metrics:** Accuracy, Precision, Recall, Specificity, and F1 Score
- **Charts:**
  1) Train VS. Validation Loss
  2) Train VS. Validation AUC
  3) Confusion Matrix

```
def create_charts(cnn, cnn_model):

## DEFINE ##

## Define 1: train & validation loss

train_loss = cnn_model.history['loss']

val_loss = cnn_model.history['val_loss']
```

```
## Define 2: train & validation AUC

train_auc_name = list(cnn_model.history.keys())[3]

val_auc_name = list(cnn_model.history.keys())[1]

train_auc = cnn_model.history[train_auc_name]

val_auc = cnn_model.history[val_auc_name]

## Define 3: y_pred & y_true

y_true = test_generator.classes

Y_pred = cnn.predict_generator(test_generator, steps = len(test_generator))

y_pred = (Y_pred > 0.5).T[0]

y_pred_prob = Y_pred.T[0]

## PLOT ##

fig = plt.figure(figsize=(13, 10))

## PLOT 1: TRAIN VS. VALIDATION LOSS

plt.subplot(2,2,1)

plt.title("Training vs. Validation Loss")

plt.plot(train_loss, label='training loss')

plt.plot(val_loss, label='validation loss')

plt.xlabel("Number of Epochs", size=14)

plt.legend()

## PLOT 2: TRAIN VS. VALIDATION AUC

plt.subplot(2,2,2)

plt.title("Training vs. Validation AUC Score")

plt.plot(train_auc, label='training auc')
```

```
plt.plot(val_auc, label='validation auc')

plt.xlabel("Number of Epochs", size=14)

plt.legend()

## PLOT 3: CONFUSION MATRIX

plt.subplot(2,2,3)

# Set up the labels for in the confusion matrix

cm = confusion_matrix(y_true, y_pred)

names = ['True Negatives', 'False Positives', 'False Negatives', 'True Positives']

counts = ['{0:0.0f}'.format(value) for value in cm.flatten()]

percentages = ['{0:.2%}'.format(value) for value in cm.flatten()/np.sum(cm)]

labels = [f'{v1}\n{v2}' for v1, v2 in zip(names, percentages)]

labels = np.asarray(labels).reshape(2,2)

ticklabels = ['Normal', 'Pneumonia']

# Create confusion matrix as heatmap

sns.set(font_scale = 1.4)

ax = sns.heatmap(cm, annot=labels, fmt='', cmap='Oranges', xticklabels=ticklabels,
yticklabels=ticklabels )

plt.xticks(size=12)

plt.yticks(size=12)

plt.title("Confusion Matrix") #plt.title("Confusion Matrix\n", fontsize=10)

plt.xlabel("Predicted", size=14)

plt.ylabel("Actual", size=14)

#plt.savefig('cm.png', transparent=True)

## PLOT 4: ROC CURVE
```

```
plt.subplot(2,2,4)

fpr, tpr, thresholds = roc_curve(y_true, y_pred_prob)

auc = roc_auc_score(y_true, y_pred_prob)

plt.title('ROC Curve')

plt.plot([0, 1], [0, 1], 'k--', label = "Random (AUC = 50%)")

plt.plot(fpr, tpr, label='CNN (AUC = {:.2f}%)'.format(auc*100))

plt.xlabel('False Positive Rate', size=14)

plt.ylabel('True Positive Rate', size=14)

plt.legend(loc='best')

#plt.savefig('roc.png', bbox_inches='tight', pad_inches=1)

## END PLOTS

plt.tight_layout()

;

## Summary Statistics

TN, FP, FN, TP = cm.ravel() # cm[0,0], cm[0, 1], cm[1, 0], cm[1, 1]

accuracy = (TP + TN) / np.sum(cm) # % positive out of all predicted positives

precision = TP / (TP+FP) # % positive out of all predicted positives

recall = TP / (TP+FN) # % positive out of all supposed to be positives

specificity = TN / (TN+FP) # % negative out of all supposed to be negatives

f1 = 2*precision*recall / (precision + recall)

stats_summary = '[Summary Statistics]\nAccuracy = {:.2%} | Precision = {:.2%} | Recall = {:.2%} | Specificity = {:.2%} | F1 Score = {:.2%}'.format(accuracy, precision, recall, specificity, f1)

print(stats_summary)
```

Results via metrics and charts

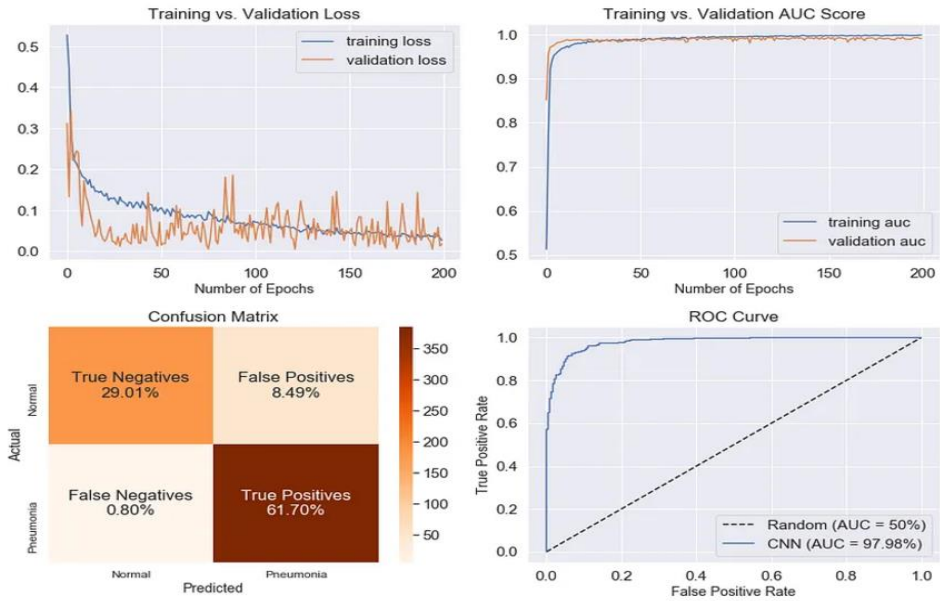
OUTPUT:

```
[Summary Statistics]
Accuracy = 90.71% | Precision = 87.90% | Recall = 98.72% |
Specificity = 77.35% | F1 Score = 93.00%
```

These metrics are:

1. **Accuracy (90.71%)** - The overall percentage of correctly classified instances out of the total instances.
2. **Precision (87.90%)** - The percentage of true positive predictions out of all positive predictions made by the model, indicating reliability in positive results.
3. **Recall (98.72%)** - The percentage of actual positive cases correctly identified by the model, reflecting its sensitivity.
4. **Specificity (77.35%)** - The percentage of true negative cases correctly identified, indicating the model's ability to avoid false positives.
5. **F1 Score (93.00%)** - The harmonic mean of precision and recall, providing a balanced evaluation of the model’s performance, especially in cases of imbalanced datasets.

These metrics collectively highlight that the model performs well, with high recall and F1 scores, making it effective for identifying positive cases like pneumonia. However, relatively lower specificity suggests room for improvement in distinguishing negative cases.



Metrics and charts of the final model

The image presents performance metrics and visualizations for a machine learning model used in pneumonia detection:

1. **Training vs. Validation Loss (Top Left)**:
   - Shows the reduction in both training and validation loss over 200 epochs.
   - Both curves stabilize at a low loss value, indicating the model effectively learns without overfitting.
2. **Training vs. Validation AUC Score (Top Right)**:
   - Tracks the Area Under the Curve (AUC) for training and validation sets during training.
   - High and consistent AUC values approaching 1.0 signify excellent classification performance.
3. **Confusion Matrix (Bottom Left)**:
   - Breaks down the model's predictions:
     - **True Negatives (29.01%)**: Correctly classified normal cases.
     - **False Positives (8.49%)**: Normal cases misclassified as pneumonia.
     - **False Negatives (0.80%)**: Pneumonia cases misclassified as normal.
     - **True Positives (61.70%)**: Correctly classified pneumonia cases.

4. **ROC Curve (Bottom Right)**:
    - Plots the true positive rate vs. false positive rate.
    - The model achieves an AUC of 97.98%, indicating highly reliable predictions compared to random guessing (AUC = 50%).

These metrics and charts highlight the model's robustness in detecting pneumonia with high sensitivity and specificity.